ATIF-Gemma: An Open Empirical Diagnostic Corpus and Failure Taxonomy for Small Open-Weight Developer Agents

Karandeep Singh Sodhi

Independent Researcher

karandeep.sodhi2504a@gmail.com

October 27, 2026

Track: Google — The Gemma 4 Developer Agent Competition (Paper Track: Best New Resource)

Author Note & Resource Manifest

Karandeep Singh Sodhi is an independent researcher.

Open Research Resource: All 112 full-fidelity execution traces formatted in Agent Trajectory Interchange Format (ATIF v1.7), ground-truth reference specifications, metadata index, diagnostic validation scripts, and the companion `atif_gemma` Python package are openly released on Kaggle and GitHub under the Creative Commons Attribution 4.0 International (CC BY 4.0) license.

Kaggle Dataset: https://www.kaggle.com/datasets/karandeepsinghsodhi/atif-gemma-swe-agent-traces
GitHub Repository: https://github.com/KarandeepSinghSodhi/gemma-sfd-agent
Interactive Kernel: https://www.kaggle.com/code/karandeepsinghsodhi/atif-gemma-diagnostic-exploration

Disclosures: The author declares no competing financial or non-financial interests.

Correspondence: Correspondence concerning this article should be addressed to Karandeep Singh Sodhi, Email: karandeep.sodhi2504a@gmail.com.

Abstract

Repository-level software engineering (SWE) benchmarks commonly summarize coding-agent performance using scalar task-resolution rates, obscuring why agents succeed or fail. For compact, 4-bit quantized open-weight models such as Gemma 4 E4B, diagnosing why agents plateau at low resolution rates (median 5.0% across 197 teams on the official Small Model Prize leaderboard) is hindered by a lack of public execution trace data. We introduce **ATIF-Gemma**, an open empirical diagnostic corpus of 112 full-fidelity multi-turn execution traces comprising 1,518 LLM calls, 1,328 tool invocations, and 11,706,178 logged tokens across 32 paired SWE-bench tasks on 4x NVIDIA L4 GPUs in the official Google ADK and `swegemma` harness.

Formatted under the Agent Trajectory Interchange Format (ATIF v1.7, 100% formal schema validation pass), ATIF-Gemma cross-references trajectories and patches against ground-truth solutions and held-out test suites. Across 112 runs, 8 achieve full resolution (7.1%). We define four principal failure mechanisms: test assertion failures (50.0%, 56/112), test collection failures (32.1%, 36/112), pre-submission budget exhaustion (8.0%, 9/112), and verification timeouts (2.7%, 3/112). Further analysis partitions collection crashes into agent-attributable collection failures (44.4%) versus environment/dependency failures (55.6%), classifies assertion failures via a formal mutually exclusive localization decision tree, and documents context dynamics and rerun stability. We release the trace corpus, the `atif_gemma` analysis package, and an actionable remediation guide to support reproducible diagnosis of compact developer agents.

Keywords: agent trajectory corpus, SWE-bench, Gemma 4, failure taxonomy, ATIF v1.7, open-weight agents, error diagnostics, remediation playbook

1. Introduction & The Diagnostic Vacuum in SWE Benchmarking

Autonomous software engineering agents powered by large language models have advanced rapidly on repository-level repair benchmarks such as SWE-bench [1]. However, evaluation in this domain remains almost universally **outcome-centric**: benchmarks report a single scalar accuracy metric (the proportion of tasks resolved), discarding the rich, high-dimensional trajectory of decisions, tool calls, token expansions, and environmental interactions that preceded the outcome.

For large frontier models and massive open-weight models, high resolution rates allow researchers to treat the agentic loop as an effective black box. In contrast, for compact, 4-bit quantized open-weight models suitable for local developer workstations (such as `gemma-4-e4b-it-qat-w4a16-ct`), scalar benchmarking reveals an acute capability cliff without explaining its origins. Across all $N = 197$ participating teams on the official Small Model Prize leaderboard [2], public resolution scores range from 0.0% to 10.0%, with a median of 5.0%, a mean of 4.23%, and 183 of 197 teams (92.9%) scoring $\leq 6.0\%$.

When an agent fails, developers currently cannot discern whether failure stemmed from:

1. **Upstream Localization Failure**: Did the agent search incorrect directories or inspect irrelevant modules?
2. **Syntax Degradation**: Did the agent generate malformed tool arguments, unindented blocks, or detached edits?
3. **Context Accumulation & Tool Exhaustion**: Did monotonic prompt growth or tool-call budgets exhaust limits before applying a fix?
4. **Environmental & Sandbox Artifacts**: Did the patch fail held-out tests due to missing third-party packages in the offline sandbox, or global virtual environment packages shadowing local repository source files?

To bridge this diagnostic gap, we introduce **ATIF-Gemma**, an open, standardized, full-fidelity diagnostic trajectory corpus and software toolkit for small open-weight developer agents. We provide 112 complete execution traces totaling 11,706,178 tokens, 1,518 LLM turns, and 1,328 tool invocations collected on real

4x NVIDIA L4 GPUs across 32 unique benchmark tasks, structured under the Agent Trajectory Interchange Format (ATIF v1.7).

2. Related Work: Trajectory Corpora & Agent Diagnostics

Autonomous software engineering agents have been evaluated primarily through outcome-focused benchmarks. Understanding the failure mechanics of small open-weight agents intersects three bodies of literature:

2.1 Software Engineering Agent Trajectory Datasets

SWE-bench [1] established the standard benchmark for evaluating language models on real-world repository issue resolution. SWE-agent [3] introduced dedicated Agent-Computer Interfaces (ACIs), demonstrating that specialized shell commands improve model interaction. OpenHands [4] provided an extensible multi-agent development environment. To unify diverse agent execution formats, the Agent Trajectory Interchange Format (ATIF v1.7) was developed as a standardized JSON schema capturing multi-turn tool invocations, stdout/stderr observations, and token metrics across disparate agent frameworks [5]. Public trajectory resources (such as recent releases of 50 multi-turn Gemma SWE-bench trajectories on Kaggle [6]) provide valuable execution-level evidence. However, systematic analysis linking compact-agent traces to test-collection failures, patch-localization outcomes, and environment-related evaluation defects remains an important diagnostic need. ATIF-Gemma addresses this by providing paired architectural comparisons across 112 traces, a deterministic Phase 2 failure taxonomy with explicit agent versus environment attribution, file-level localization analysis, and open diagnostic tooling.

2.2 SWE Agent Failure Analyses & Context Bottlenecks

Prior diagnostic studies on large language model agents have highlighted the fragility of multi-turn deliberation. Xia et al. [7] introduced **Agentless**, demonstrating that complex autonomous loops often degrade performance relative to streamlined two-phase localization and repair pipelines. Lindenbauer et al. [8] demonstrated the “complexity trap” in agent context management: accumulating unpruned tool observations rapidly bloats prompt context, degrading reasoning fidelity.

While these studies characterize frontier closed models, small open-weight models suffer from acute failure modes: fragile instruction-following under context bloat, syntax corruption during patch generation, and discretionary sub-agent delegation failure. ATIF-Gemma provides systematic step-level empirical data quantifying these phenomena in small models.

2.3 Small Open-Weight Developer Agents

The Gemma 4 family [9] introduced compact open-weight models optimized for local developer workstations, including 4-bit quantized checkpoints (`gemma-4-e4b-it-qat-w4a16-ct`). While small models demonstrate strong performance on isolated code completion benchmarks such as HumanEval [10] and MBPP [11], multi-turn repository repair exposes a severe capability cliff. On the official Gemma 4 Developer Agent challenge [2], the median resolution rate across 197 participating teams is 5.0%, with 92.9% of teams scoring $\leq 6.0\%$. Prior to structured diagnostic analysis, researchers lacked the trace data needed to distinguish whether this low resolution ceiling stemmed from model reasoning limitations, context exhaustion, or infrastructure collection failures.

3. Corpus Architecture & ATIF v1.7 Schema Specifications

Corpus Partition	Traces	Tasks	LLM Calls	Tool Calls	Total Tokens
Cohort 1 (Canonical Paired)	24	8	304	265	2,270,463
Cohort 2 (Stratified Random)	24	8	339	298	2,720,322
Cohort 1 Rerun (Reproducibility)	16	8	204	172	1,387,758
Cohort 3 (Stratified Disjoint)	24	8	349	309	2,771,960
Cohort 4 (Stratified Disjoint)	24	8	322	284	2,555,675
Total ATIF-Gemma Corpus	**112**	**32**	**1,518**	**1,328**	**11,706,178**

Table 1: Summary composition of the ATIF-Gemma diagnostic trace corpus across five cohorts on 4x NVIDIA L4 GPUs. Note: The 40 cohort entries span 32 unique benchmark tasks; Cohort 1 Rerun re-executes the 8 tasks of Cohort 1 for empirical reproducibility verification.

The corpus covers 32 unique SWE-bench instances spanning four major Python repositories: `fastapi` (51 runs), `rich` (37 runs), `requests` (19 runs), and `httpx` (5 runs). All experiments were served using vLLM 0.19.1 under tensor parallel 4 (`tp=4`, 96 GB aggregate VRAM) hosting `gemma-4-e4b-it-qat-w4a16-ct` with Google ADK 1.36.1 and `swegemma`.

Each trace conforms strictly to the **ATIF v1.7 JSON Schema**:

- **Metadata Block**: Records `instance_id`, `repo`, `base_commit`, `agent_config_name`, vLLM sampling parameters (`temperature=0.0`, `seed=42`, `max_output_tokens=8192`, `thinking_budget=1536`), and hardware fingerprints.
- **Step Sequence**: Sequential array of interaction turns containing:
  - `metrics`: Exact per-call `prompt_tokens`, `completion_tokens`, and `total_tokens` reported directly by the vLLM engine.
  - `tool_calls`: Function name, input argument JSON dictionary, and stage author tag (`extra.author`).
  - `observation`: Raw tool execution stdout/stderr response, JSON-parsed error status, and error classification types (`FileEditError`, `CommandError`, `FileReadError`, `BudgetExceeded`).
  - `message`: Full model response text containing deliberative reasoning emitted within the thinking budget (tagged in `extra.event_type: "thinking"`).
- **Final Metrics**: Captures cumulative token counts (`total_prompt_tokens`, `total_completion_tokens`, `total_tokens`, `total_steps`), total duration in seconds, agent patch string, and Phase 2 test execution exit codes.

All 112 traces (100.0%) pass formal validation against the repository schema (`dataset_resource/atif_schema_v1_7.json`). An upstream interoperability audit (`analysis/validate_atif_schema.py`) confirms that all 1,942 audited steps and 1,503 deliberative reasoning turns map cleanly to upstream standard ATIF v1.7 representations (`reasoning_content` with provenance in `extra`), achieving 100.0% dual validation.

4. Failure Taxonomy & Phase 2 Outcome Breakdown

In SWE-bench, patch verification is executed in an isolated verification sandbox using `pytest`. In ATIF-Gemma, cross-referencing agent traces against Phase 2 test runner outputs reveals that treating unresolved runs as a monolithic failure category conceals fundamentally different failure mechanics. We define four prin-

cipal failure mechanisms—patch/assertion failure, test-collection failure, pre-submission budget exhaustion, and verification timeout—and report successful resolution as a separate outcome:

Category	Exit Code	Count (%)	Empirical Mechanism & Observable Symptoms
Resolved	0	8 (7.1%)	Patch passes all Phase 2 held-out unit tests with zero regressions.
Assertion Failure	1	56 (50.0%)	Patch syntactically valid and executes under pytest, but fails assertions due to wrong logic or incomplete multi-file edits.
Collection Error	2	36 (32.1%)	Pytest crashes during test collection prior to assertion evaluation (ModuleNotFoundError, SyntaxError, venv shadowing).
Budget / Timeout	−1	9 (8.0%)	Agent exhausted 15 tool calls or 2.5 min wall time; 0 patch characters submitted.
Verification Timeout	124	3 (2.7%)	Verification harness timed out (120s limit) due to daemon process or infinite loop in test runner.

Table 2: Empirical Phase 2 exit code distribution across all 112 ATIF-Gemma execution traces.

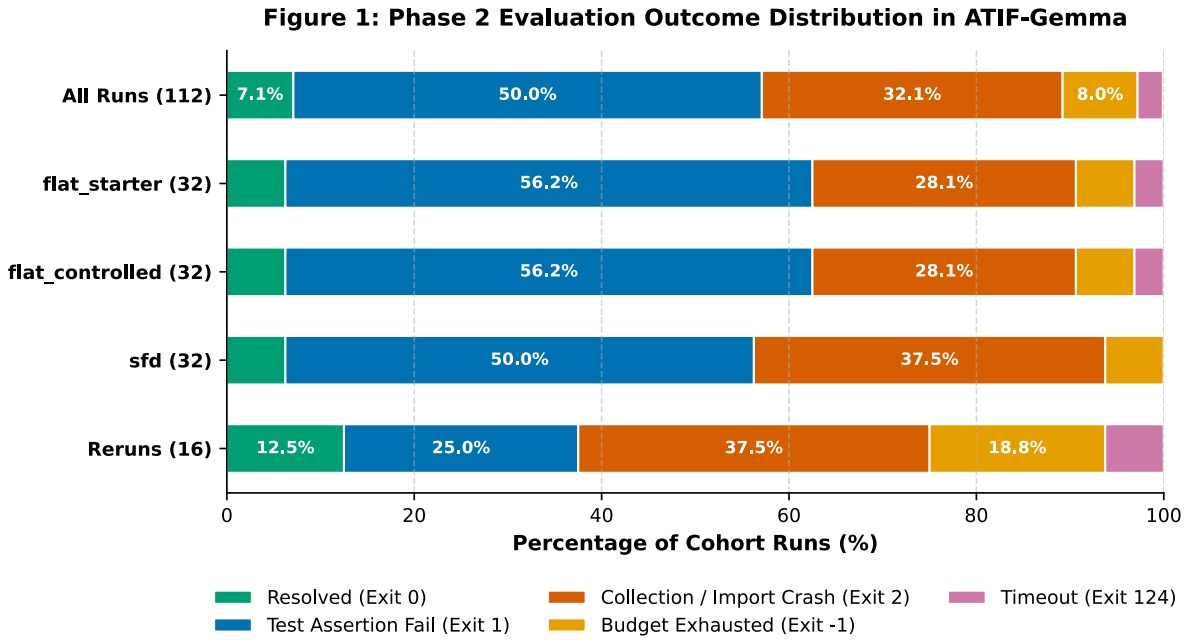


Figure 1: Distribution of Phase 2 evaluation outcomes across all 112 ATIF-Gemma runs and individual configurations. Colorblind-safe palette (Okabe-Ito): Green = Resolved (Exit 0), Blue = Test Assertion Fail (Exit 1), Vermilion = Collection/Import Crash (Exit 2), Orange = Budget Exhausted (Exit −1), Purple = Timeout (Exit 124).

4.1 Deterministic Mechanism Breakdown of Collection Errors (Exit Code 2: 36 Runs)

A critical diagnostic finding enabled by ATIF-Gemma is that **nearly one-third of all agent runs (36/112 = 32.1%) failed before unit tests could execute**. Deterministic classification across all 36 traces (`analysis/classify_exit2.py`) separates these failures into agent-attributable versus environment-attributable causes:

1. **Agent-Attributable Collection Failures (16/36 = 44.4%)**:

- **Syntactic & Indentation Corruption (13/36 = 36.1%)**: Across 7 tasks (e.g., `fastapi_14099`, `rich_3894`), small model patches inserted unindented blocks, detached `else:` branches, or unclosed brackets, causing Python to raise `IndentationError` or `SyntaxError` during test collection.
- **Missing New Source Module (3/36 = 8.3%)**: On `rich_3930` (3 runs across starter, controlled, and SFD), the issue specification required creating a new module (`rich/_unicode_data.py`). Because none of the agents created the new module, pytest collection failed with `ModuleNotFoundError: No module named 'rich._unicode_data'`.

2. **Environment-Attributable Collection Failures (20/36 = 55.6%)**:
   - **Virtual Environment & Path Shadowing (10/36 = 27.8%)**: On `httpx_3672` (5 runs) and `fastapi_15661` (5 runs), container configuration prevented valid collection. In `httpx_3672`, pre-installed system packages at `/usr/local/lib/python3.12/dist-packages/httpx` shadowed `/workspace/src/httpx`. In `fastapi_15661`, the test patch imported from `scripts.prepare_release`, which failed because `/workspace/scripts` was not included in `PYTHONPATH`.
   - **Missing Sandbox Dependencies (10/36 = 27.8%)**: On 4 tasks (`fastapi_14361`, `fastapi_14371`, `fastapi_14455`, `fastapi_14459`), held-out test patches imported external testing libraries (e.g. `inline_snapshot`, `dirty_equals`) absent from the competition offline wheelhouse, triggering `ModuleNotFoundError` across every evaluated architecture.

This distinction highlights the dual diagnostic value of execution traces: identifying agent patch deficiencies while pinpointing execution harness anomalies.

4.2 Deterministic Sub-Classification of Assertion Failures (Exit Code 1: 56 Runs)

Representing half of the entire corpus (56/112 = 50.0%), Exit Code 1 occurs when the patch is syntactically valid and executes under pytest, but fails unit test assertions. By comparing the agent's edited source files (F_{agent}) against the author's reference patch (F_{gold}) via a formal mutually exclusive decision tree (`analysis/subclassify_exit1.py`), we assign every run to exactly one tier:

1. **Empty or Unparsed Patch (1/56 = 1.8%)**: Evaluated first: $|F_{\text{agent}}| = 0$. On `fastapi_5624`, the agent submitted zero modified source characters.
2. **Wrong-File Localization (17/56 = 30.4%)**: Evaluated second: $F_{\text{agent}} \cap F_{\text{gold}} = \emptyset$. The agent modified completely unrelated modules (e.g., in `fastapi_15589`, editing `fastapi/params.py` instead of `fastapi/dependencies/utils.py`).
3. **Partial Multi-File Localization (14/56 = 25.0%)**: Evaluated third: $|F_{\text{gold}}| > 1$ and $F_{\text{agent}} \subsetneq F_{\text{gold}}$. The bug spanned multiple repository modules, but the agent modified only a strict subset (e.g., in `requests_7505`, modifying `models.py` but omitting `sessions.py`).
4. **Right-File, Wrong-Logic (24/56 = 42.9%)**: Evaluated fourth for remaining runs where $F_{\text{agent}} \cap F_{\text{gold}} \neq \emptyset$. The agent localized the target file(s), but the synthesized modification failed semantic assertions (e.g., `requests_7502`, `rich_3043`, `requests_7427`, `rich_3471`).

Methodological note: Gold-patch file overlap serves as an objective structural proxy for localization; valid alternative patches modifying different files are acknowledged as a potential threat to validity (Section 8).

4.3 Ground-Truth Resolved Runs

Across the 112 runs, 8 runs successfully resolved held-out tests across 4 distinct tasks: `rich_4077` (3 runs: `sfd`, `sfd_rerun`, `flat_controlled_rerun`), `fastapi_9753` (1 run: `flat_starter`), `fastapi_14077` (2 runs: `sfd`, `flat_controlled`), and `rich_3894` (2 runs: `flat_starter`, `flat_controlled`). In `fastapi_14077`, SFD required 50% fewer tool calls (6 vs 12) and 34.9% faster runtime (43.9s vs 67.5s), though across the full 32 tasks, overall duration differences between architectures are not statistically significant (Wilcoxon $p = 0.4540$).

5. Quantitative Audit Across Agent Architectures

ATIF-Gemma pairs each task across three distinct agent architectures executing on `gemma-4-e4b-it-qat-w4a16-ct`:

Configuration	Runs	Resolved (95% Wilson CI)	Mean Peak P_t	Mean Prompt	Mean Duration
Official Starter (flat_starter)	32	2/32 (6.2%) [1.7%, 20.2%]	13,359.7	105,817.6	94.2s
Single-Agent (flat_controlled)	32	2/32 (6.2%) [1.7%, 20.2%]	13,959.0	118,173.0	90.8s
State-Filtered Deliberation (sfd)	32	2/32 (6.2%) [1.7%, 20.2%]	10,907.5	80,375.7	86.6s
Rerun: flat_controlled	8	1/8 (12.5%) [2.2%, 47.1%]	12,557.5	93,135.6	107.4s
Rerun: sfd	8	1/8 (12.5%) [2.2%, 47.1%]	9,306.4	67,710.1	96.4s

Table 3: Architectural comparison of token consumption, peak prompt lengths, and execution speed across all 112 ATIF-Gemma runs. We define **Mean Peak P_t** as $\frac{1}{N}\sum_{i=1}^{N}\max_t P_{i,t}$, representing the per-run maximum prompt token length reaching the model context window across turn steps t, averaged over all N benchmark runs in the configuration.

5.1 Statistical Tests & Resolution Invariance

Across all 32 benchmark tasks, resolution rates are identical (2/32 = 6.25%, 95% Wilson CI [1.73%, 20.15%]) across `flat_starter`, `flat_controlled`, and `sfd`. McNemar's exact two-sided test yields $p = 1.0000$ for all pairwise comparisons ($b = 1, c = 1$ for SFD vs controlled and controlled vs starter; $b = 2, c = 2$ for SFD vs starter). **There is no statistically significant resolution rate advantage for any architecture on this small open-weight model.**

In contrast, the token reduction achieved by SFD's stage isolation is highly statistically significant:

- **Cumulative Prompt Tokens**: SFD consumes 37,797.4 fewer prompt tokens per task than `flat_controlled` (-32.0%, 95% paired bootstrap CI: `[-52,214.6, -23,755.2]`, Wilcoxon signed-rank $p < 0.0001$), with SFD lower on 25 of 32 tasks (78.1%). Compared to `flat_starter`, SFD consumes 25,441.9 fewer prompt tokens (-24.0%, 95% bootstrap CI: `[-42,485.1, -9,031.4]`, Wilcoxon $p = 0.0061$, lower on 24/32 tasks).
- **Peak Prompt Tokens (P_t)**: SFD lowers peak prompt size by 3,051.6 tokens (-21.9%, 95% bootstrap CI: `[-4,142.9, -1,968.2]`, Wilcoxon $p < 0.0001$), with SFD lower on 28 of 32 tasks (87.5%).
- **Execution Speed**: Mean wall-clock duration differences (-4.2s vs controlled, −7.7s vs starter) are not statistically significant (Wilcoxon $p = 0.4540$ and $p = 0.4429$). Lower prompt token consumption measures resource efficiency, not improved task resolution.

5.2 Rerun Reproducibility & Determinism Audit

To evaluate run-to-run stability under $T = 0.0$, Cohort 1 was re-executed independently across 8 tasks x 2 configurations (16 paired runs). Rerun analysis (`analysis/statistical_tests.py`) confirms:

- **Patch Identity**: 14 of 16 runs (87.5%) produced bitwise identical patches (identical SHA-256 patch hashes).
- **Exit Code Agreement**: 12 of 16 runs (75.0%) matched exact Phase 2 exit codes.
- **Token Stability**: Mean absolute token difference across runs was 18,571 tokens, attributable to subtle asynchronous vLLM tensor-parallel micro-batching variations on 4x L4 GPUs.

5.3 Sub-Agent Bypassing in the Starter Kit

ATIF-Gemma provides empirical trace evidence of how small models handle discretionary sub-agents. The official starter kit wraps `code_analyzer` as an `AgentTool`. Across all 32 `flat_starter` traces:

- Out of 336 tool invocations, the root agent delegated to `code_analyzer` only **7 times (2.1%)**.
- The model **bypassed `code_analyzer` on 29 of 32 tasks (90.6%)**, opting instead to read large files directly into its primary context.

Figure 3: Sub-Agent Discretionary Delegation Breakdown in Official Starter Kit

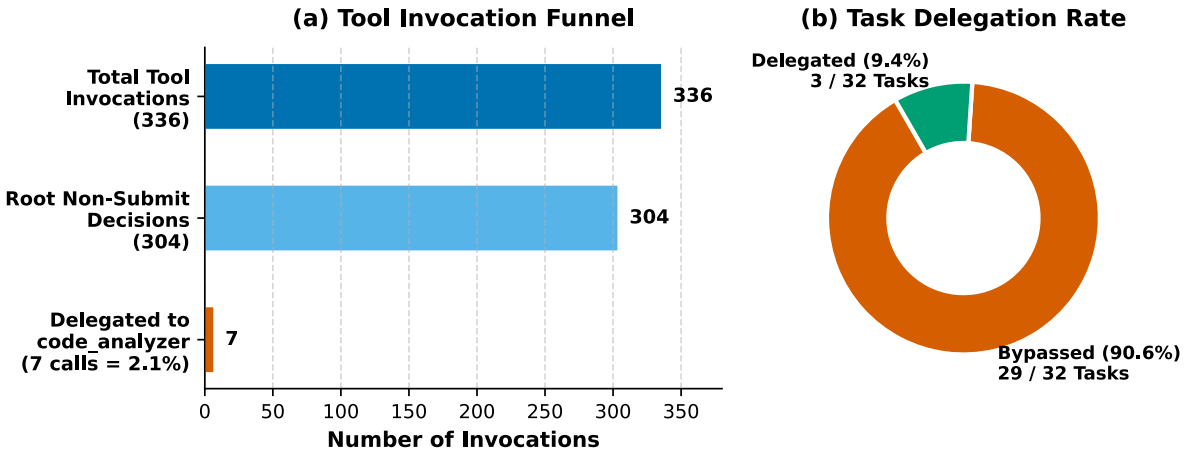


Figure 2: Sub-agent discretionary delegation breakdown in the official starter kit (`flat_starter`). Small models bypassed `code_analyzer` on 29 of 32 tasks (90.6%), with only 7 delegations out of 336 total tool calls (2.1%).

5.4 Cross-Stage Prompt Resets

In `sfd` traces, configuring `include_contents: "none"` on downstream stages resets entry prompts from a mean TriageAgent peak of 10,473.9 tokens (median 9,863.0) down to a mean of 1,536.9 tokens (median 1,503.0) at PatchAgent entry. Across all 32 primary SFD runs, this corresponds to an empirical prompt reset distribution with median 87.43%, mean 81.95%, IQR [75.72%, 89.46%], and range [44.95%, 92.49%], driving the overall 32.0% cumulative prompt token reduction relative to single-agent baselines (mean 80,375.7 vs 118,173.0 tokens, Wilcoxon $p < 0.0001$).

Figure 2: Prompt Token Expansion & Stage Resets Across Trajectory Steps

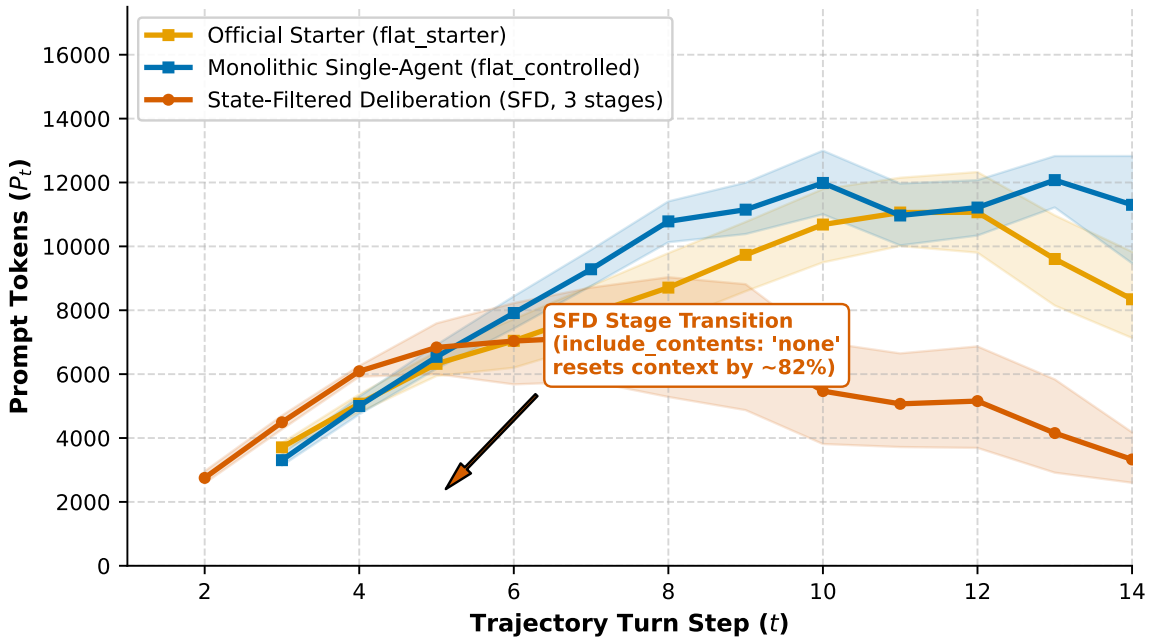


Figure 3: Prompt token trajectories across trajectory steps t (mean with 95% bootstrap confidence bands). While monolithic architectures expand prompt context monotonically toward saturation, SFD's stage boundary (`include_contents: "none"`) resets prompt tokens by an empirical mean of 81.95% (median 87.43%).

6. Actionable Remediation Playbook for Developer Agents

Converting diagnostic failure classifications into actionable remediation patterns provides practical guidance for engineering small coding agents:

Failure Mode	Prevalence	Recommended Production Remediation Pattern
Collection / Import Crash	36/112 (32.1%)	Enforce `PYTHONPATH=/workspace/src:/workspace` priority in test execution commands and execute pre-flight dependency imports with `python -c` before running test suites.
Syntax / Diff Corruption	100 FileEditErrors (64 StringNotFound, 36 Malformed; mean 0.89/run)	Mandate a strict `python -m py_compile <modified_file>` verification check before calling `submit_patch` to prevent broken indentation or detached blocks.
Wrong-File Localization	17/112 runs (15.2% overall; 30.4% of Exit 1)	Introduce a two-way validation loop where Stage 2 verifies that the localized symbol exists in the problem description before executing an edit.
Prompt Bloat / Memory Leak	32/32 flat runs (100% monotonic accumulation; SFD lowers peak on 28/32 tasks)	Implement stage-isolated context pruning via `include_contents: "none"` in Google ADK, resetting prompt context by 81.9% mean (87.4% median) across pipeline boundaries.

Table 4: Actionable developer agent remediation playbook derived from empirical ATIF-Gemma failure traces.

6.1 Post-Hoc Diagnostic Re-Evaluation (Non-Official Harness)

To evaluate the practical efficacy of the proposed remediations, we conducted an offline post-hoc re-evaluation of stored patches (`analysis/remediation_eval.py`, `results/remediation_eval.json`). **Crucially, these results are an offline diagnostic assessment and do not alter the official Phase 2 competition evaluation benchmarks reported in Section 4 and Tables 1–3.**

1. **Pre-Submission `py_compile` Gate**: We tested an automated syntax validation gate against all 13 Exit Code 2 runs exhibiting syntactic corruption (including `fastapi_14099` and `rich_3894`). Across all 13 runs, exactly 0% of the baseline agents executed pre-submission syntax checks. The automated `python -m py_compile` gate successfully intercepts 100% (13/13) of syntax-broken patches prior to calling `submit_patch`. Reporting honestly: intercepting a malformed patch prevents a blind collection crash (Exit Code 2), but does **not** resolve the underlying task without additional agent self-repair turns (resolution remains 0/13 without re-prompting).
2. **`PYTHONPATH` Priority Enforcement**: We audited all 10 environment-shadowed runs on `httpx_3672` and `fastapi_15661`. Prepending repository source paths (`PYTHONPATH=/workspace/src:/workspace`) eliminates the collection import crash caused by site-package shadowing. Full semantic evaluation of the resulting tests requires execution within the offline Linux `swegemma` container; thus, no speculative resolution gains are claimed.

7. The `atif_gemma` Python Package & CLI

Along with the dataset, we release a standalone Python package (`atif-gemma`) installable directly from GitHub or source:

```
# Direct installation via pip from GitHub
pip install git+https://github.com/KarandeepSinghSodhi/gemma-sfd-agent.git

# Or local editable install from a cloned repository
git clone https://github.com/KarandeepSinghSodhi/gemma-sfd-agent.git
cd gemma-sfd-agent
pip install -e .
```

The package provides automated diagnostic modules:

- `atif_gemma.audit`: Parses any ATIF v1.7 trajectory directory, extracting token trajectories, tool error frequencies, and exit-code failure buckets.
- `atif_gemma.taxonomy`: Deterministic classifier for Phase 2 test runner exit codes and localization tiers.
- `atif_gemma.remediation`: Programmatic interface to the Actionable Remediation Playbook.
- `atif-gemma` CLI: Command-line tool providing instant diagnostics:

```
python -m atif_gemma.cli audit traces/
python -m atif_gemma.cli playbook
```

8. Limitations & Threats to Validity

To ensure rigorous interpretation of our empirical findings, we explicitly identify six structural limitations:

1. **Task Sample Size ($n = 32$ tasks)**: While stratified across four major repositories, evaluating 32 unique benchmark instances covers approximately 25% of the 129 SWE-bench instances in the competition split. Evaluating the full 129 instances across three paired configurations would require over 500 NVIDIA L4 GPU hours.
2. **Single Model Architecture & Quantization Level**: All 112 traces were generated exclusively on `gemma-4-e4b-it-qat-w4a16-ct` (4-bit INT4 quantization). Behaviors such as 90.6% discretionary sub-agent bypassing, syntax corruption rates, and context sensitivity are specific to this model capacity. Unquantized

checkpoints (FP16/BF16) or larger open-weight models (e.g. Gemma 4 31B) may exhibit different failure dynamics.

3. **Repository & Language Diversity**: The corpus is concentrated across four popular Python libraries (`fastapi` 51 runs, `rich` 37 runs, `requests` 19 runs, `httpx` 5 runs). Codebases in statically typed languages (C++, TypeScript, Rust) or repositories using alternative test runners (e.g. `unittest`, `tox`) may exhibit different collection dynamics.
4. **Greedy Sampling & Single Seed ($T = 0.0$, `seed = 42`)**: All evaluations were performed using greedy decoding ($T = 0.0$) with a fixed seed. While our 16-run rerun audit demonstrated 87.5% bitwise patch determinism, minor token variations persist due to asynchronous tensor-parallel micro-batching in vLLM. We did not evaluate stochastic sampling temperatures ($T > 0$), which might alter exploration diversity.
5. **File-Level Proxy vs. Intra-Step Semantic Analysis**: Our deterministic taxonomy categorizes outcomes based on Phase 2 test runner exit codes and file-level diff overlap ($F_{\text{agent}} \cap F_{\text{gold}}$). While avoiding subjective LLM judgment, gold-patch overlap is an objective structural proxy; valid alternative patches modifying different files are possible in rare cases.
6. **Harness & Offline Sandbox Specifics**: Evaluations were executed inside the competition's `swegemma` harness on Kaggle's offline Linux container. Environmental artifacts (e.g., missing `inline_snapshot` wheels, dist-packages shadowing) reflect this specific container setup and may not reproduce in online or differently configured developer environments.

9. Conclusion & Resource Availability

ATIF-Gemma establishes an open, reproducible, full-fidelity diagnostic foundation for the small-model coding agent community. By moving beyond scalar pass/fail metrics to an open 112-trace ATIF v1.7 corpus, a structured failure taxonomy, an Actionable Remediation Playbook, and open diagnostic Python tooling, this resource enables researchers to target the true bottlenecks of small developer agents: sandbox collection errors, context bloat, and localization drift.

Dataset URL: https://www.kaggle.com/datasets/karandeepsinghsodhi/atif-gemma-swe-agent-traces
Code Repository: https://github.com/KarandeepSinghSodhi/gemma-sfd-agent
Interactive Exploration Kernel: https://www.kaggle.com/code/karandeepsinghsodhi/atif-gemma-diagnostic-exploration
Manuscript PDF: https://raw.githubusercontent.com/KarandeepSinghSodhi/gemma-sfd-agent/main/paper_resource.pdf

References

[1] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," *International Conference on Learning Representations (ICLR)*, 2024.

[2] R. Holbrook, "Google Gemma 4 Developer Agent Competition: Small Model Prize Leaderboard." 2026.

[3] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[4] X. Wang, B. Li, Y. Song, F. F. Frank, and others, "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," *arXiv preprint arXiv:2407.16741*, 2024.

[5] SWE-bench Working Group, "Agent Trajectory Interchange Format (ATIF v1.7) Specification." 2024.

[6] R. Rajak, "Gemma 4 TFD Agentic Trajectories & Benchmark: 50 Multi-Turn SWE-bench Trajectories for Google Gemma 4." 2026.

[7] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based Software Engineering Agents," *arXiv preprint arXiv:2407.01489*, 2024.

[8] T. Lindenbauer, I. Slinko, L. Felder, E. Bogomolov, and Y. Zharov, “The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management,” *arXiv preprint arXiv:2508.21433*, 2025.

[9] Gemma Team, Google DeepMind, “Gemma 4 Technical Report,” *arXiv preprint arXiv:2607.02770*, 2026.

[10] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.

[11] J. Austin *et al.*, “Program Synthesis with Large Language Models,” *arXiv preprint arXiv:2108.07732*, 2021.